

in the *exampledocs* folder. You may look at the structure of each of the documents that get posted.

There are multiple ways for sending data to be indexed:

- **XML-post:** Solr allows for a predefined set of XML schemas, which specify the instruction or the action to be performed. This may range from the simple addition of new documents, to deletion, updation and committing of data. These XML commands can simply be POSTed to the server (at <http://localhost:8080/solr/update> in this case), and it will perform the specified operation. This is precisely what the *post.jar* application does. For example, to delete a document, you can run the following command in a terminal:


```
curl http://localhost:8080/solr/update -F stream.body='
<delete><id>VAG02B</id></delete>'
```
- **CSV:** Solr can import the data to be accepted in the form of a CSV format, where values are usually separated by a delimiter such as a comma or a tab. It can also be set up to accept streams of data in the form of CSV, if your data source generates it. CSV data can be sent to Solr by sending data to <http://localhost:8080/solr/update/csv>. A sample *books.csv* file is included in *exampledocs*, which can be uploaded using curl in a manner similar to that shown above.
- **Solr Cell:** What more could you possibly want from a search server, than its ability to index PDF, Word, Excel, JPEG and even MP3 files? Solr Cell, a.k.a. *ExtractingRequestHandler*, is what allows you to do so! It has been built as a wrapper around the Apache Tika project, which can strip out data from a wide range of supported documents—and what's more, it can even auto-detect the MIME type of the data.
- **DataImportHandler:** This has been built to handle direct database imports, either with the help of JDBC, or XML imports from a URL or a file. The XML import functionality can be used to index data over HTTP/REST APIs, or even RSS feeds. In addition to full import, it supports delta-imports—that is, a partial update of the index, reflecting only the recent changes in the database since the last build, instead of a complete rebuild of the index. The *DataImportHandler* can be defined in the *solrconfig.xml* file; it can further point to a file for detailed configuration of the connection and data source information. The *example-DIH* folder already contains some examples, along with detailed documentation, for you to try out.

Querying

So finally we have all our data set, and we are ready to make a query or two in order to know how it works. Let's make a basic query now: type *'solr'* into the search box at the admin



Figure 2: Solr Admin Console

console, and see what you get. The URL generated looks something like what's shown below:

<http://localhost:8080/solr/select?q=solr&version=2.2&start=0&rows=10&indent=on>

We can guess that <http://localhost:8080/solr/select> is the endpoint at which we will be making our queries, while the *q=solr* parameter represents our search query. The *start=0* and *rows=10* parameters are used for paginating, since Solr does not display all results at once. Besides this, there are a lot of parameters that have been left out, which include features like boosting, sub-expressions, filter queries, sorting, and of course, faceting! Additionally, the *DisMax* Query parser allows for even more sophisticated queries that should be more than enough to satisfy even advanced users.

Looking at the search results, we'll be able to see XML that has been enclosed within the `<response>` tags. The initial metadata, like the status, query time, etc, are enclosed within the `<lst name="responseHeader">` element, and the subsequent results are enclosed within the `<result name="response" numFound="1" start="0">` element, which further contains the `<doc>` element that represents each document returned as a result. It should be obvious that the *numFound* attribute represents the number of results returned.

In conclusion, it will be useful to note that add-on modules for using Solr with CMSes like Drupal and Wordpress are already available. These modules include all the configuration information to set up Solr for your website, and provide a much better searching mechanism than the one included with the core. **END** 

Further reading

- [1] Solr Wiki—<http://wiki.apache.org/solr/FrontPage>
- [2] Indexing Data in Solr 1.4 Enterprise Search Server—<http://bit.ly/1DNtbf>
- [3] Search Smarter with Apache Solr—<http://bm.co/31OQmr>

By: Ankit Mathur

The author is a 21-year old geek. He has a crush on Java, and also loves flirting with almost anything related to Web technologies. Feel free to poke fun at this article, and direct your feedback to ankitreloaded@gmail.com.